

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 5)

Softwaretest: Terminologie und Methoden

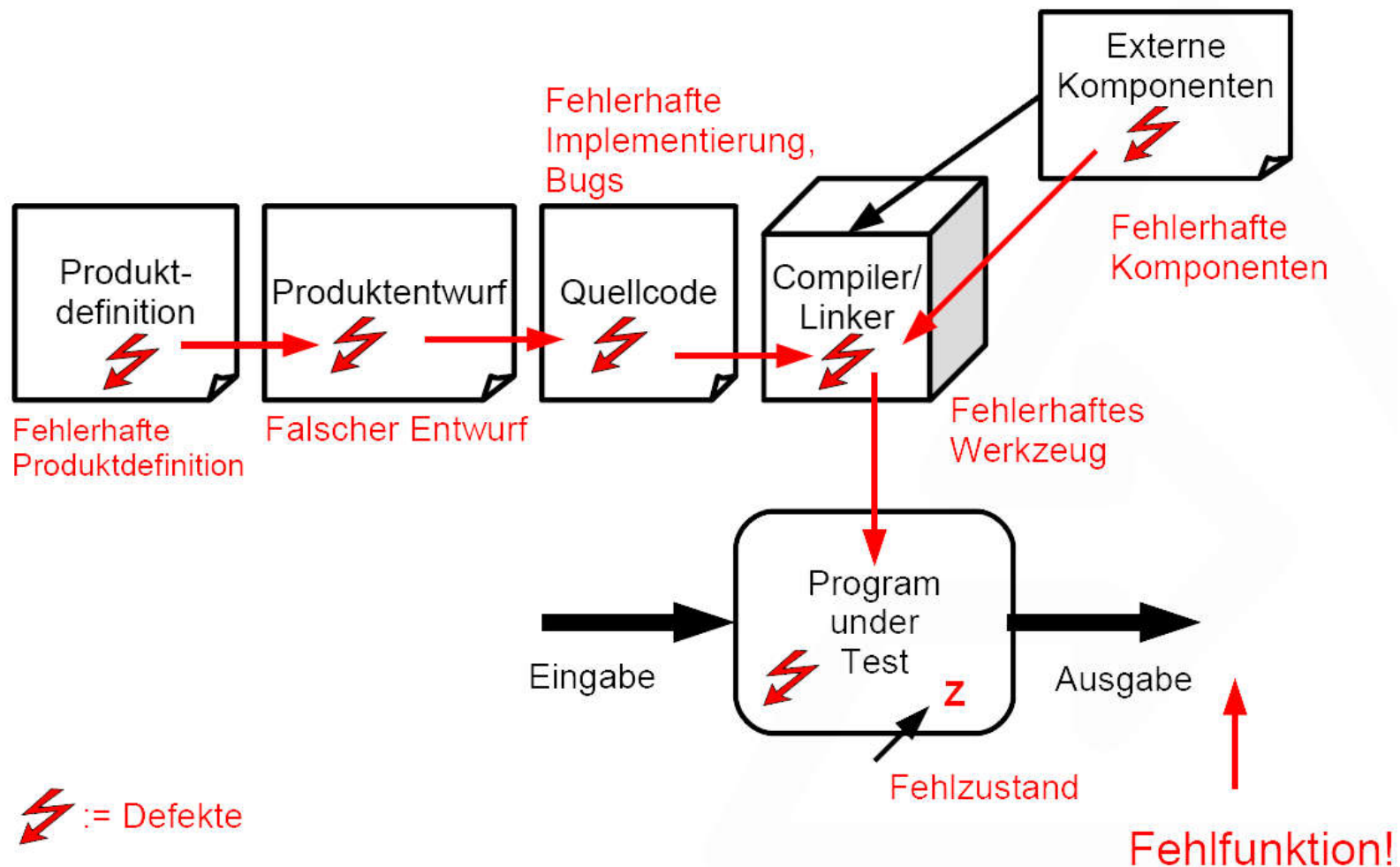
Definition: Software-Test

Software Testing ist der Prozess, in dem ein System oder eine Komponente **ausgeführt** werden, die Ergebnisse beobachtet oder **aufgezeichnet** werden und schließlich ein Aspekt des Systems oder der Komponente **bewertet** wird.
(nach IEEE Std 610.12-1990)

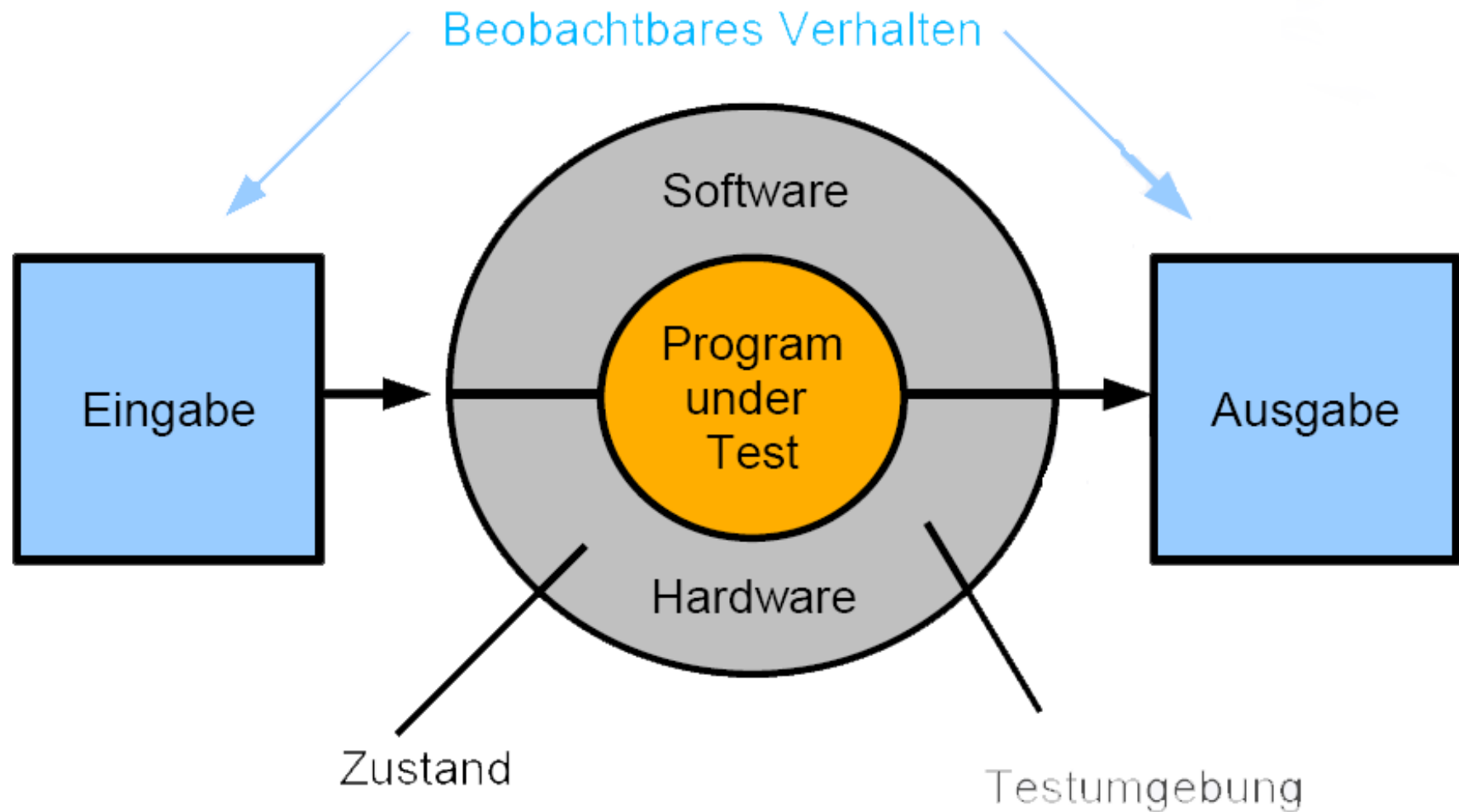
- Relativ
- Bestandteil des Entwicklungsmodells
- Wichtig für Projekterfolg

"Program testing can be used to show the presence of bugs, but never to show their absence!" – E.W. Dijkstra

Mögliche Fehlerquellen

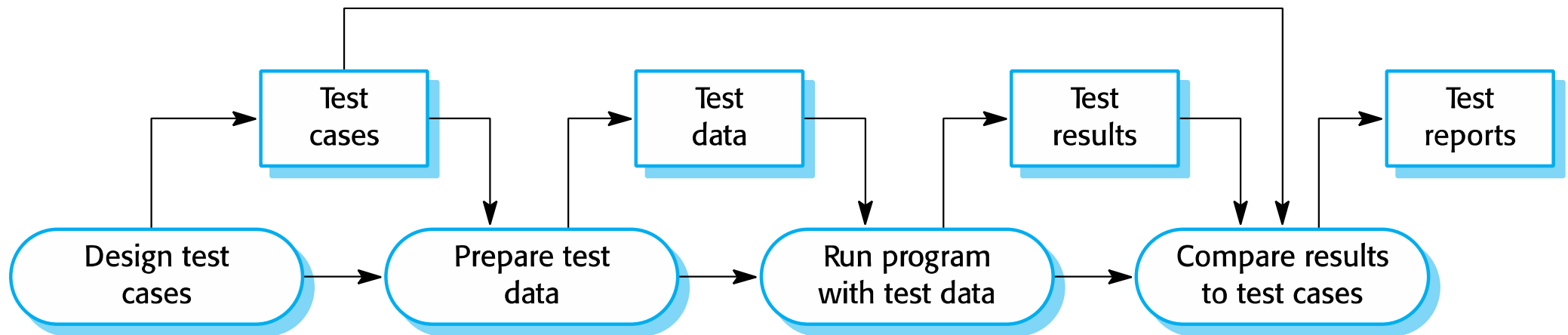


Testumgebung



Fehlerfreiheit muss aus dem beobachtbaren Verhalten geschlossen werden

Allgemeines Modell des Test-Prozesses



Testetappen

- **Entwicklungstest:** Das System wird während der Entwicklung auf Fehler und Defekte getestet
- **Releasetest:** Ein eigenständiges Test-Team testet eine vollständige Version des Systems vor der Veröffentlichung
- **Anwendertest:** Anwender (tatsächliche oder potentielle) testen das System in ihrer eigenen Umgebung (-> „Beta-Tester“)

Welche Testverfahren gibt es?

- Smoke Test
- Black-Box Testing
- White-Box Testing
- Grey-Box Testing
- Stress Testing
- Regression Testing

Smoke Test



- Analogie zu: Man schließt etwa eine neue Maschine testweise an Stromnetz an und prüft, ob sie überhaupt etwas tut oder direkt „in Qualm aufgeht“
- Bestanden, wenn kein „Rauch“ aufsteigt
- Bei Software:
 - Einfaches, manuelles Testen
 - Oberflächliches Ausführen grundlegender Funktionen
- „Rauch“ ist:
 - Exception
 - Stacktrace
 - Fehlermeldung
 - Offensichtliches Fehlverhalten (etwa: startet erst gar nicht)

Black-Box Testing

- Testmethode ohne Kenntnisse um innere Funktionsweise des „Program under Test“
- Fokus auf den in der Spezifikation festgelegten Funktionalitäten und nicht deren Implementierung
- Automatisierungsgrad ist häufig minimal
 - Bei graphischer Benutzeroberfläche: Aufwändiger zu automatisieren
 - Häufig großer organisatorischer Aufwand (z.B. mit separater Testabteilung)
- Beispiele von Varianten:
 - Unstrukturiertes Ausprobieren (etwa durch Anwender)
 - Strukturiertes, dokumentiertes Ausführen nach Testplänen

White-Box Testing

Manchmal auch „Glass-Box“ statt „White-Box“

- Testmethode mit Kenntnissen um innere Funktionsweise des „Program under Test“
- Fokus der Fehleraufdeckung liegt auf den Software Komponenten und nicht auf der zugrundeliegenden Spezifikation
- Verfügbarkeit des Quellcodes erforderlich
- Automatisierungsgrad ist gegeben
- Geringer organisatorischer Aufwand

Black-Box: Vergleich zu White-Box

Vorteile Black-Box

- bessere Verifikation des **Gesamtsystems** als mit White-Box-Tests
- Überprüfen der **Spezifikation**
- **neutral**, kein Testen „um Fehler herum“
- Testen von **semantischen** Eigenschaften bei geeigneter Spezifikation
- **Portabilität** von systematisch erstellten Testsequenzen auf plattformunabhängige Implementierungen

Nachteile Black-Box

- größerer organisatorischer Aufwand
- Test zusätzlich eingefügter Funktionen bei Implementierung nur durch Zufall
- **Spezifikationsqualität**: Muss hinreichend gut sein
- **soziale Aspekte**

Grey-Box Testing

- Verbindung von Black-Box und White-Box basierten Testen durch testgetriebene Entwicklung („Test First“)
- Entwickler erstellen Tests für das „Program Under Test“ (White-Box) lediglich mit dem Wissen um die spezifizierten Anforderungen (Black-Box)
- Erfordert hohes Maß an Disziplin (Rigor) und Erfahrung
- Kein vollständiger Ersatz für Black-Box Testing

Stress Testing

- Testen, ob das „Program under Test“ auch unter großen Belastungen korrekt funktioniert
 - Kriterien wie beispielsweise *Zeitanforderungen*, *Volumenanforderungen*, *Zuverlässigkeit*
- Beispiele:
 - z.B. Client/Server Anwendungen:
 - Hält der Server auch der Belastung durch sehr viele Clients stand?
 - Wieviele Clients kann er bedienen?
 - Wie verringert sich die Antwortzeit im Verhältnis zur Anzahl Clients?
 - Große Datenmengen:
 - Wieviele Daten kann das „Program under Test“ verarbeiten?

Regressionstests

- lateinisch regredior, regressus sum = *zurückschreiten*
- Erkennt Situation: „Gestern ging's noch!“
- Vorhandene, funktionierende und getestete Programmteile werden darauf getestet, dass sie auch weiterhin funktionieren und nicht durch Weiterentwicklungen oder Modifikationen zerstört wurden
 - Automatisierte Tests, die regelmäßig die bereits erreichte Funktionalität sicherstellen
 - Wichtig bei großen Softwareprojekten, die ständig weiterentwickelt werden
 - Nachgelagert und zusätzlich Funktionalitätstests

"Also as a consequence of the introduction of new bugs, program maintenance requires far more system testing per statement written than any other programming. Theoretically, after each fix one must run the entire batch of test cases previously run against the system, to ensure that it has not been damaged in an obscure way. In practice, such regression testing must indeed approximate this theoretical idea, and it is very costly."

-- Fred Brooks, The Mythical Man Month

Principles of Software Testing nach Bertrand Meyer

Principle 1: Definition

To test a program is to try to make it fail.

Principles of Software Testing nach Bertrand Meyer

Principle 2: Tests Versus Specs

Tests are no substitute for specifications.

Principles of Software Testing nach Bertrand Meyer

Principle 3: Regression Testing

Any failed execution must yield a test case, to remain a permanent part of the project's test suite.

Principles of Software Testing nach Bertrand Meyer

Principle 4: Applying Oracles

Determining success or failure of tests must be an automatic process.

Principles of Software Testing nach Bertrand Meyer

Principle 5: Manual and Automatic Test Cases

An effective testing process must include both manually and automatically produced test cases.

Zusammenfassung

- Testen ist ein wesentlicher Bestandteil der Verifikation und Validierung von Software
- Testen spielt in jeder Entwicklungsphase eine bedeutsame Rolle
- Verschiedenste Techniken und Methoden unterstützen die Effektivität des Testens

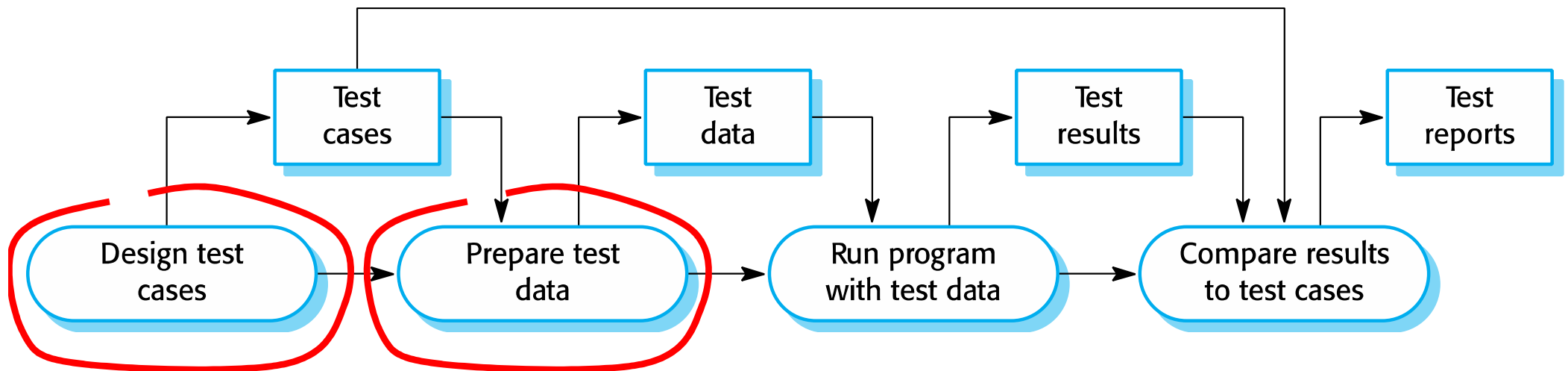


SOFTWARE-QUALITÄT UND SICHERHEITSFragen (TEIL 6)

Softwaretest: Testeingabedaten und Testabdeckung

Allgemeines Modell des Test-Prozesses

Welche Testdaten? Welche Testfälle?



Wie kommt man zu Testeingabedaten?

- **Grenzwertanalyse**
 - Grenzwerte der Definitions- / Wertebereiche von Ein- und Ausgabedaten werden getestet
 - Hypothese: diese Werte sind für das Verhalten besonders kritisch und dort werden häufig Fehler gemacht (Defekte eingefügt)
 - z.B.: Zahlendarstellungen (Überlauf), Indizes, Grenzen von Zählschleifen (Iteratoren), Division durch 0,...
- **Äquivalenzklassentest**: verwandt mit Grenzwertanalyse (→ nächste Folie)
- **Zufallsdaten**
 - Testdaten werden zufällig generiert
 - Hypothese: durch gute Wahrscheinlichkeitsverteilung der Eingabedaten wird eine gute Überdeckung des Programms erreicht
- **Use Case-bezogener Funktionstest**
 - Testen, ob die spezifizierte Funktionalität tatsächlich implementiert ist
 - Hypothese: wenn die volle Funktionalität vorhanden ist und kein Fehlverhalten produziert, ist das Programm fehlerfrei...

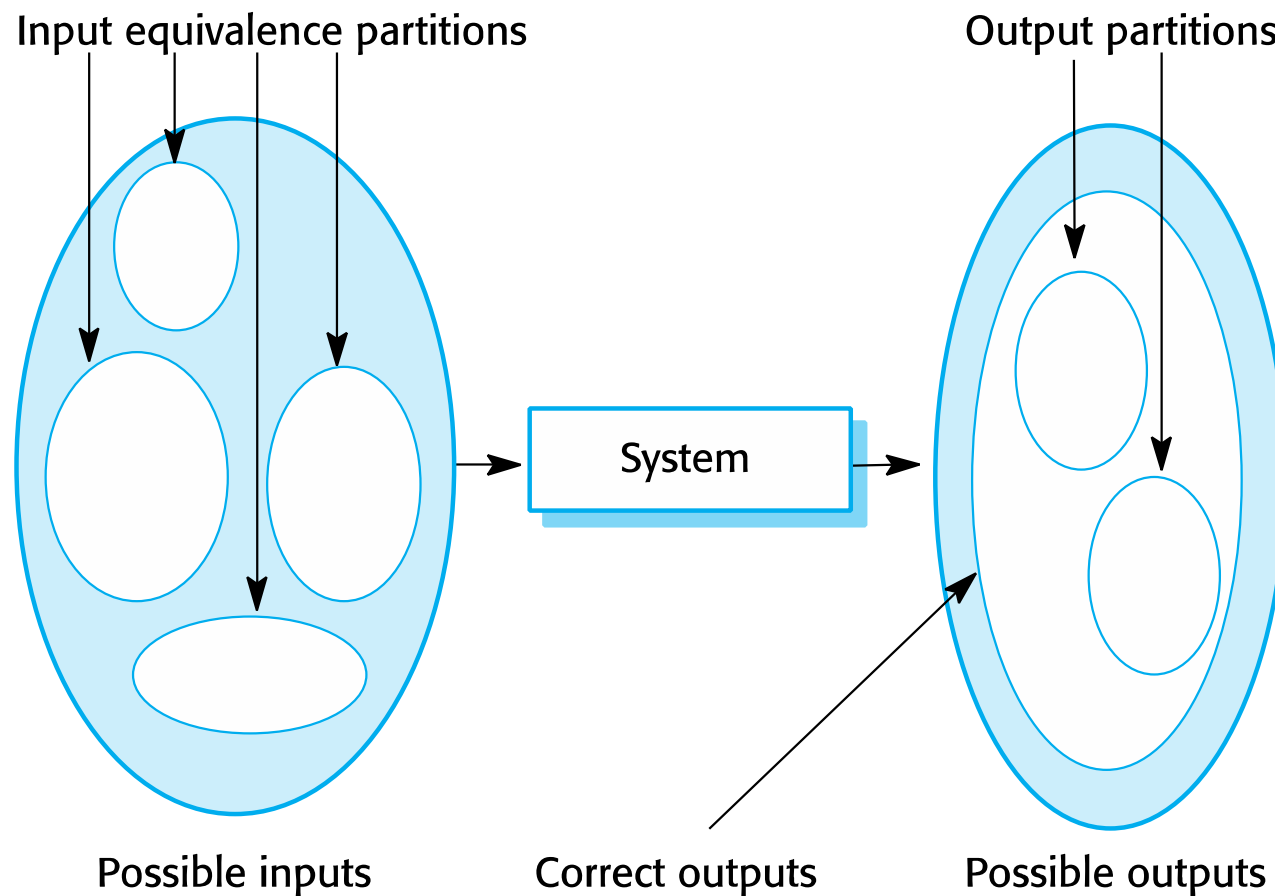
Auswahl von Testeingabedaten

Äquivalenzklassentest

- Methode zur Auswahl und Einschränkung der Eingabedaten:
- Eingabedaten und Ausgaben lassen sich oft in verschiedene Klassen mit ähnlichen Mitgliedern unterteilen
- Jede dieser Klassen ist ein Bereich, in dem das Programm sich für jedes Klassenmitglied gleich verhält
- Testfälle sollten deshalb aus jeder dieser Äquivalenz-Klassen stammen

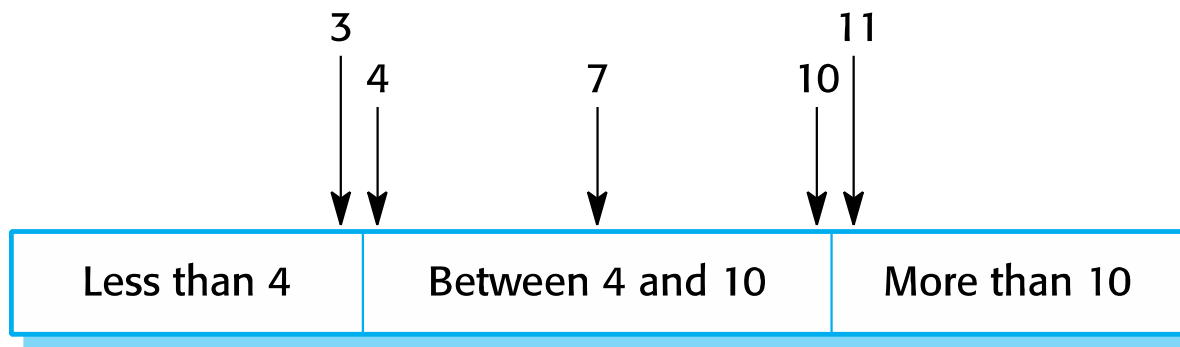
Auswahl von Testeingabedaten

Äquivalenzklassentest

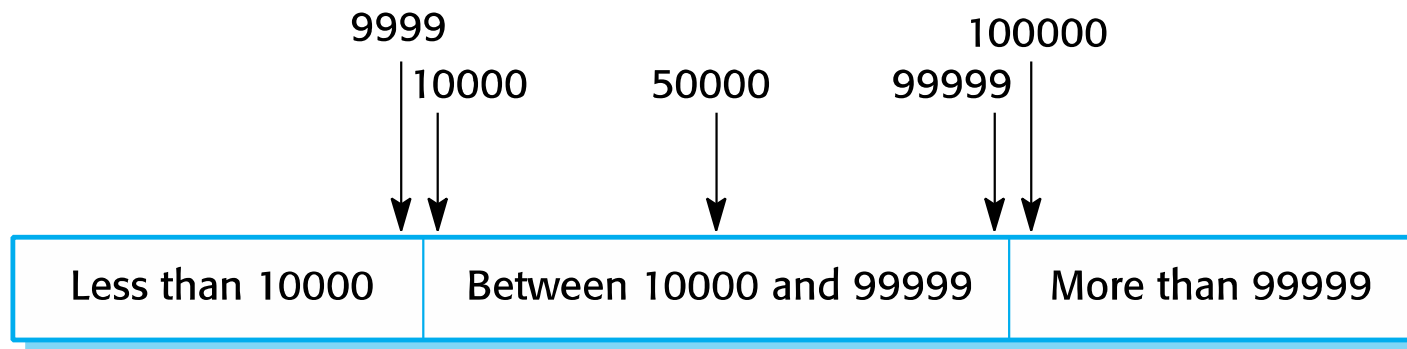


Auswahl von Testeingabedaten

Beispiel Äquivalenzklassentest



Number of input values



Input values

Jeremy

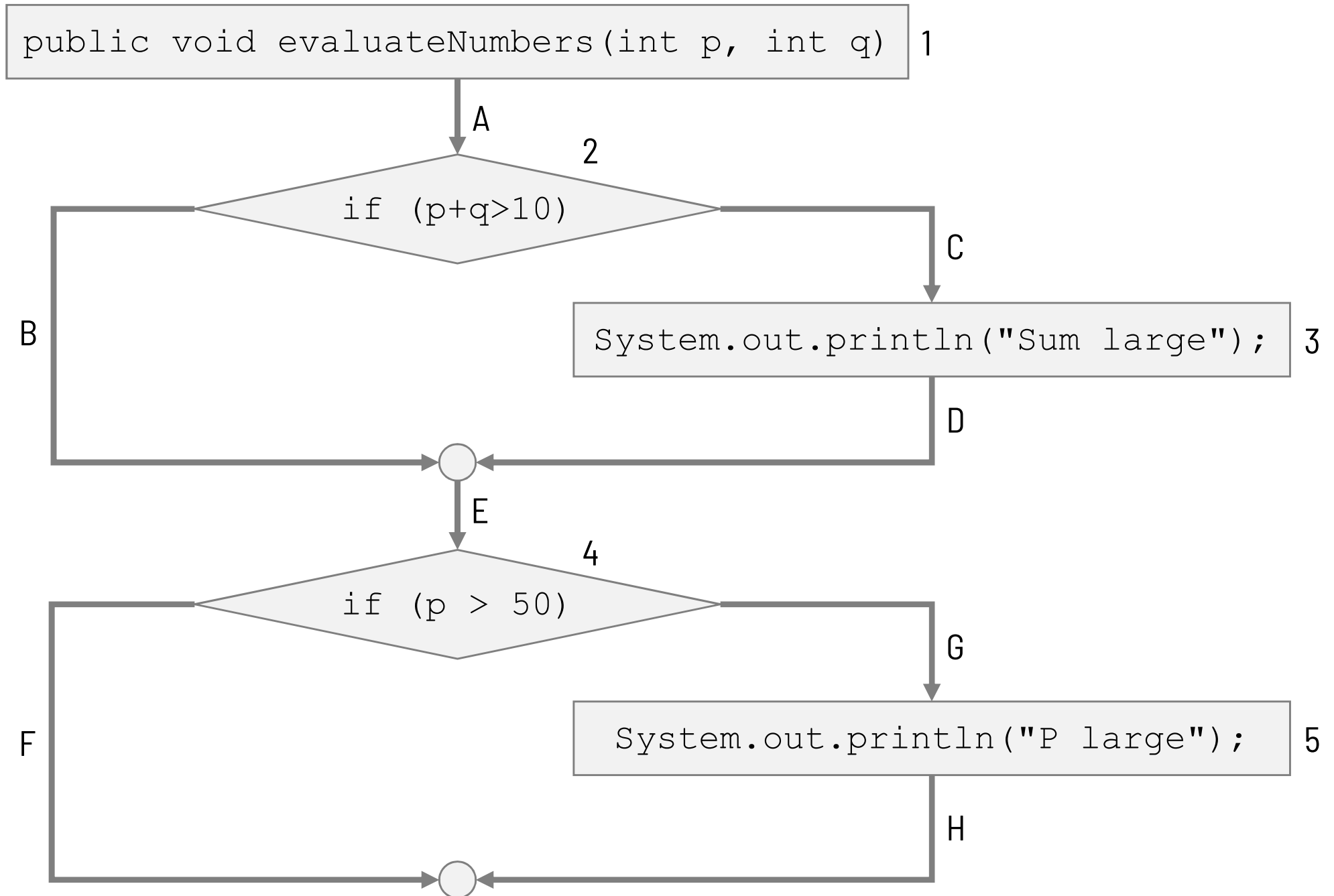
Jeremy (siehe Abbildung 4.1) möchte als Qualitätsbeauftragter seines Teams sicher gehen können, dass die Testprogramme in guter Qualität geschrieben worden sind. Wie kann er die Qualität der Testprogramme überprüfen? Wie kann er mögliche Lücken in den Testprogrammen identifizieren? Kann er dafür Werkzeuge einsetzen?



Abbildung 4.1: Jeremy ist 31 Jahre alt. Freunde beschreiben ihn als korrekt und sehr gewissenhaft. Im Beruf wird er für diese Eigenschaften sehr geschätzt. Er ist in seinem Entwicklungsteam der Qualitätsbeauftragte.

Beispiel

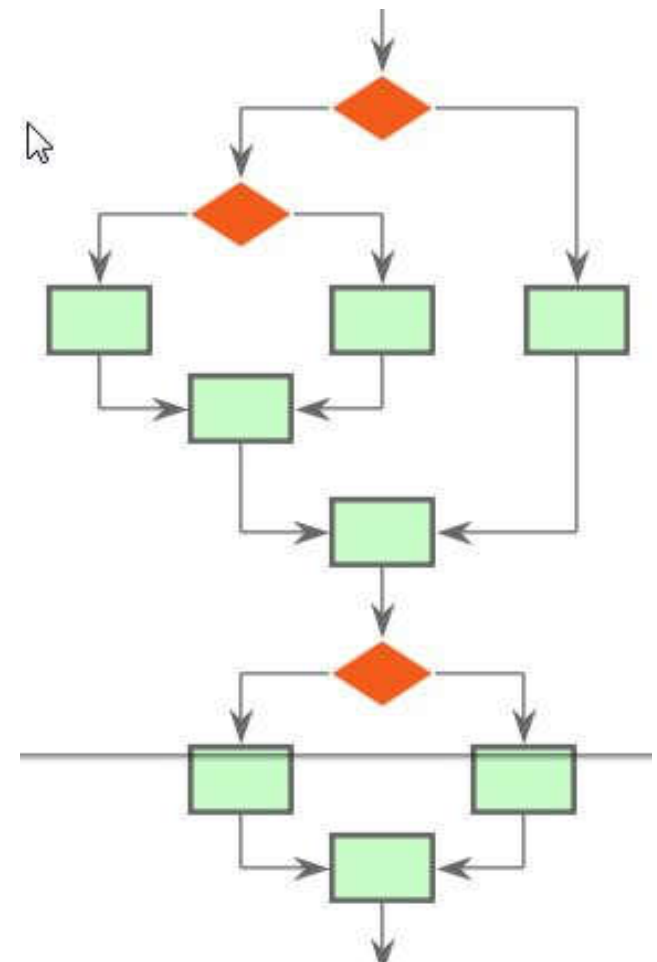
```
public void evaluateNumbers(int p, int q) {  
    if (p+q>10)  
        System.out.println("Sum large");  
    if (p > 50)  
        System.out.println("P large");  
}
```



Programmausführung als Graph

Kontrollflußgraph:

- Anweisungen als Knoten
- Kontrollfluss als gerichtete Kante
- Pfad als Sequenz von Knoten und Kanten

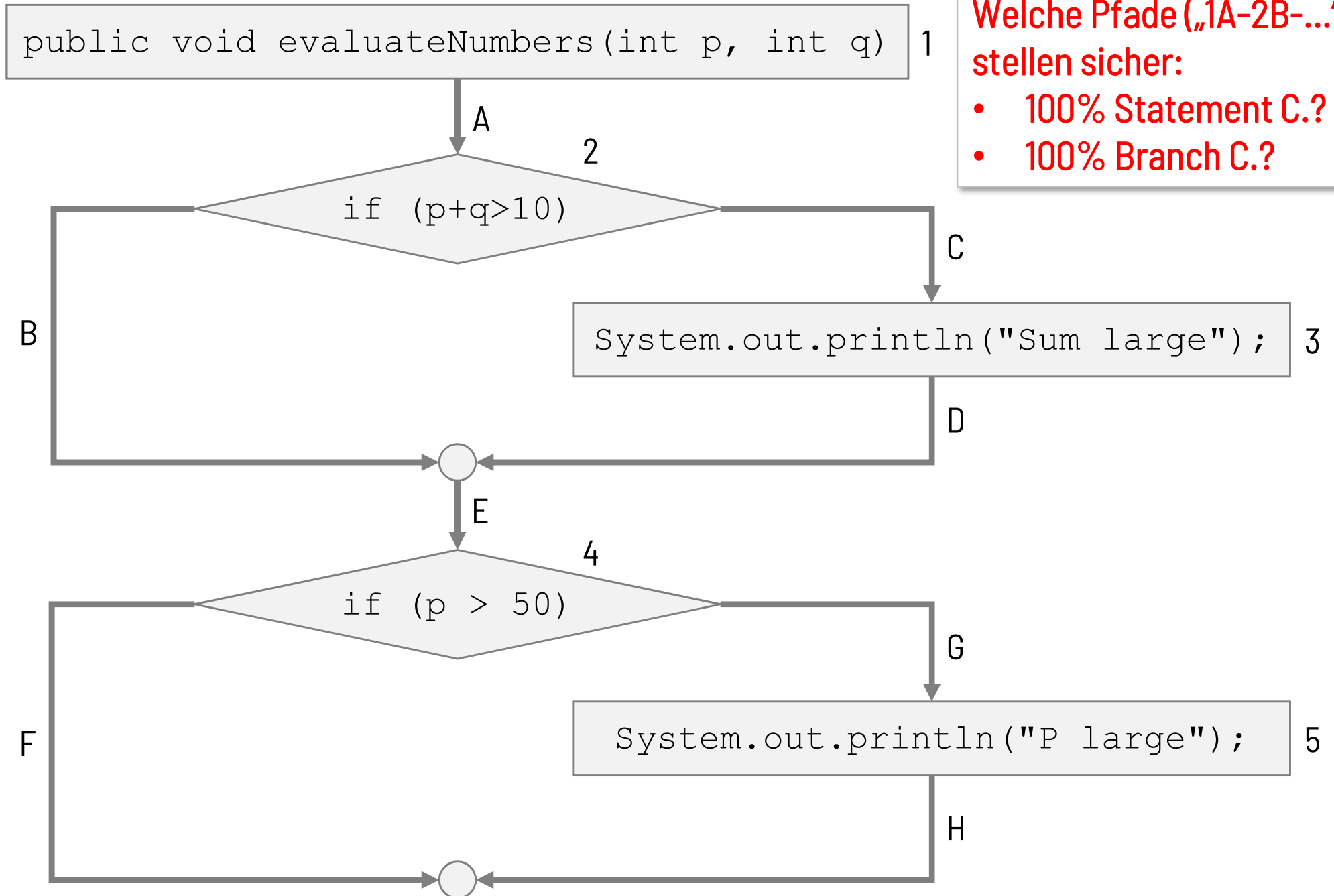


Testabdeckung (Test Coverage)

- Testabdeckung: Frage, wieviel vom Programmcode durch die Tests tatsächlich getestet wird –
wie gut ist er durch Tests abgedeckt?
- Grundlage: Kontrollfluß
- **Statement Coverage – SC**
Prozentuale Anzahl der Anweisungen („Zeilen“), die bei der Ausführung aller Tests besucht wurden
- **Branch Coverage – BC**
Prozentuale Anzahl der Verzweigungen (Ergebnisse von Entscheidungen, Eintritte und Austritte), die bei der Ausführung aller Tests besucht wurden

Testabdeckung

- Zeile im Quellcode ist **abgedeckt durch Tests**, wenn sie **mindestens einmal** durch einen Tests ausgeführt wurde
- **Ziel:** Auffinden und Minimieren von ungetesteten Codes
- **Kritik:** Aussagekraft der Metrik ist fragwürdig für hohe Werte (z.B. 95%)
- **Praxis:** Zwischen 70-90% wünschenswert
- **Werkzeuge:** Cobertura, EclEmma / JaCoCo



Zusammenhang Statement Coverage / Branch Coverage

- 100% Branch Coverage $\Rightarrow$ 100% Statement Coverage
- 100% Statement Coverage sagt nichts über Branch Coverage aus

Beispiel: Addieren zweier positiver ganzer Zahlen

```
package de.hrw.swen.unittesting;

public class PositiveZahlenAddieren
{
    public int addieren(int a, int b)
    throws RuntimeException {
        if ((a < 0) || (b < 0))
            throw new RuntimeException(
                "Negative Zahlen sind nicht"
                + " erlaubt");
        return (a + b);
    }
}
```

```
package de.hrw.swen.unittesting.test;
import static org.junit.Assert
    .assertEquals;
import org.junit.Test;
import de.hrw.swen.unittesting
    .PositiveZahlenAddieren;

public class PositiveZahlenAddierenTest
{
    @Test
    public void testAddieren2Positiven
        ...
    }

    @Test
    public void testAddieren1Negativen
        ...
    }
}
```



Übung

- Erstellen Sie einen Testfall, damit die *Statement Coverage* 100 % erreicht!
- Welche Testfälle müssen Sie erstellen, damit die *Branch Coverage* 100 % erreicht?

```
package de.hrw.swen.unittesting;  
  
public class PositiveZahlenAddieren  
{  
    public int addieren(int a, int b)  
        throws RuntimeException {  
        if ((a < 0) || (b < 0))  
            throw new RuntimeException(  
                "Negative Zahlen sind nicht"  
                + " erlaubt");  
        return (a + b);  
    }  
}
```

Beispiel: Addieren zweier positiver ganzer Zahlen

```
package de ...
public class ... {
    {
        ...
        public int ...
        throws RuntimeException {
            if ((a ...
                thro ...
                "Neg ...
                + " ...
            return ...
        }
    }
}
```

```
...
public class PositiveZahlenAddierenTest
{
    ...
    @Test
    public void testAddieren2PositiverZahlenErfolgreich() {
        // Durchführen
        int ergebnis = adder.addieren(1,2);
        // Bewerten
        assertEquals(3, ergebnis);
    }
    ...
}
```

Beispiel: Addieren zweier positiver ganzer Zahlen

```
package de ...
public class ... {
    public int ...
    throws RuntimeException {
        if ((a ...
        throw ...
        "Neg ...
        + " ...
        return ...
    }
}

...
public class PositiveZahlenAddierenTest
{
    ...
    @Test
    public void testAddieren1NegativerZahlFehler() {
        try {
            // Durchführen
            int ergebnis = adder.addieren(1, -2);
        } catch (RuntimeException e) {
            // Bewerten
            assertEquals("Negative Zahlen sind nicht erlaubt",
                e.getMessage());
            return;
        }
        fail();
    }
    ...
}
```

Beispiel: Addieren zweier positiver ganzer Zahlen

Abdeckungsanzeige (Statement Coverage)
mit EcLEmma in Eclipse

>0%, aber
<100%

100%

```
1 package de.hrw.swen.unittesting;
2
3 /**
4  * Positive ganze Zahlen sollen addiert werden.
5  *
6  * @author andriesc
7  *
8  */
9 public class PositiveZahlenAddieren {
10     public int addieren(int a, int b) throws RuntimeException {
11         if ((a < 0) || (b < 0))
12             throw new RuntimeException("Negative "
13                                     + "Zahlen sind nicht erlaubt");
14         return (a + b);
15     }
16 }
17
18
```

Beispiel: Addieren zweier positiver ganzer Zahlen – Lösung

```
...
package de

public class ...
{
    public int ...
    throws RuntimeException {
        if ((a ...
            thro ...
            "Neg ...
            + " ...
        return ...
    }
}

...
@Test
public void testAddieren1NegativerZahlFehler() {
    try {
        // Durchführen
        int ergebnis = adder.addieren(1, -2);
    } catch (RuntimeException e) {
        // Bewerten
        assertEquals("Negative Zahlen sind nicht erlaubt",
            e.getMessage());
        try {
            int ergebnis = adder.addieren(-1, 2);
        } catch (RuntimeException e1) {
            assertEquals("Negative Zahlen sind nicht erlaubt",
                e1.getMessage());
            return;
        }
        fail();
    }
    fail();
}
```

Beispiel: Addieren zweier positiver ganzer Zahlen – Lösung

Element	Coverage	Covered Instru...	Missed Instruct...	Total Instructio...
swen	 18,5 %	51	225	276
src	 18,5 %	51	225	276
de.hrw.swen.patterns.decorator	 0,0 %	0	114	114
de.hrw.swen.patterns.factorymethc	 0,0 %	0	56	56
de.hrw.swen.unittesting.test	 38,9 %	35	55	90
de.hrw.swen.unittesting	 100,0 %	16	0	16
PositiveZahlenAddieren.java	 100,0 %	16	0	16
PositiveZahlenAddieren	 100,0 %	16	0	16
addieren(int, int)	 100,0 %	13	0	13

```
8  */
9  public class PositiveZahlenAddieren {
10     public int addieren(int a, int b) throws RuntimeException {
11         if ((a < 0) || (b < 0))
12             throw new RuntimeException("Negative "
13                                     + "Zahlen sind nicht erlaubt");
14         return (a + b);
15     }
16 }
17 }
18
```

Zusammenfassung

- Verschiedene Vorgehensweisen unterstützen dabei, sinnvolle Testdaten auszuwählen
- Testabdeckung ist ein hilfreiches Werkzeug, um die Qualität der entwickelten Software erhöhen zu können
- Werkzeuge unterstützen bei der Ermittlung der Abdeckung

